PROJECT REPORT

Project Title

“Automated Cell Detection and Counting System using Python and Image Processing”

B.Tech. Semester – 02

Submitted By (Group of 5):

1. Name: Siril Bhimala

   Enrollment No: 92500111080

2. Name: Javvadi Usha Kiran

   Enrollment No: 92500111027

3. Name: Dhanvir Yandra

   Enrollment No: 92500111056

4. Name: Pala Praveen Kumar Reddy

   Enrollment No: 92500111043

Under the Guidance of: Prof.Rushi Raj Vala

Academic Year: 2026 - 2027

Head of Department

Department of ECE

Marwadi University

Signature

ABSTRACT

This project presents the development of an automated system for cell detection and counting using Python and image processing techniques. Manual counting of cells from microscopic images is a time-consuming and error-prone process, especially when dealing with large datasets. To overcome these limitations, this system uses computer vision methods to identify and count cells efficiently.

The implementation is carried out using Python libraries such as OpenCV and NumPy. The input image is first pre-processed through grayscale conversion and noise reduction techniques. Further, methods like thresholding and Hough Circle Transform are applied to detect circular structures corresponding to cells. The detected cells are then highlighted, and their count is displayed as output.

The system aims to improve accuracy, reduce human effort, and provide consistent results. It works effectively for images with clear and distinct cell boundaries, although challenges may arise in cases of overlapping or irregularly shaped cells. Overall, the project demonstrates the practical application of image processing in solving real-world biological and medical problems.

OBJECTIVES

- To develop an automated system for detecting cells from images
- To implement image processing techniques using Python and OpenCV
- To achieve accurate and efficient cell counting with minimal human effort

INTRODUCTION

In the modern era, image processing has become a powerful tool in various domains such as medical diagnostics, biological research, and industrial automation. One of the important applications of image processing is the analysis of microscopic images, particularly for detecting and counting cells. Traditionally, this task is performed manually by experts, which is not only time-consuming but also prone to human errors and inconsistencies.

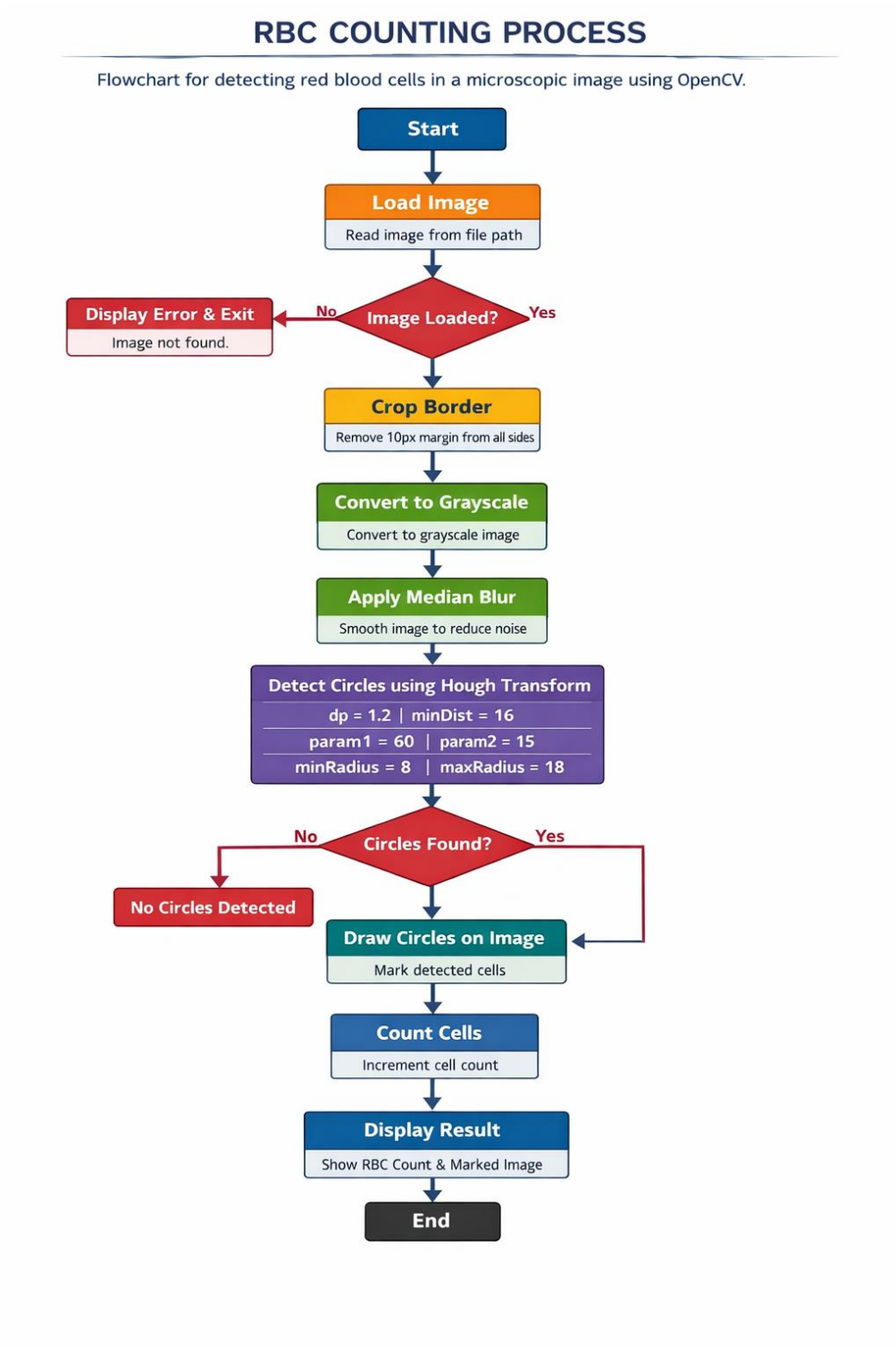
With the advancement of computer vision techniques, it is now possible to automate such processes using programming languages like Python. Python provides a wide range of libraries, such as OpenCV and NumPy, which make it easier to handle image data and apply complex algorithms efficiently. These tools enable the development of systems that can analyze images with high speed and accuracy.

In this project, an automated cell detection and counting system is developed using Python. The system processes input images by converting them into grayscale and applying noise reduction techniques to enhance image quality. Further, advanced methods like thresholding and Hough Circle Transform are used to detect circular shapes that represent cells. Once detected, the cells are marked, and their total count is calculated.

The main objective of this system is to provide a reliable and efficient solution for cell counting, reducing the need for manual intervention. It ensures consistency in results and significantly improves the speed of analysis. However, the performance of the system depends on factors such as image quality, lighting conditions, and cell overlap.

This project highlights the practical implementation of image processing concepts and demonstrates how Python can be used to solve real-world problems in science and engineering. It also lays the foundation for future improvements, such as integrating machine learning techniques for enhanced accuracy and adaptability.

Flow Chart / Block Diagram



Software Required:

- **Programming Language:** Python (Version 3.x)
- **Libraries Used:** OpenCV (cv2), NumPy
- **Development Environment (IDE):** PyCharm

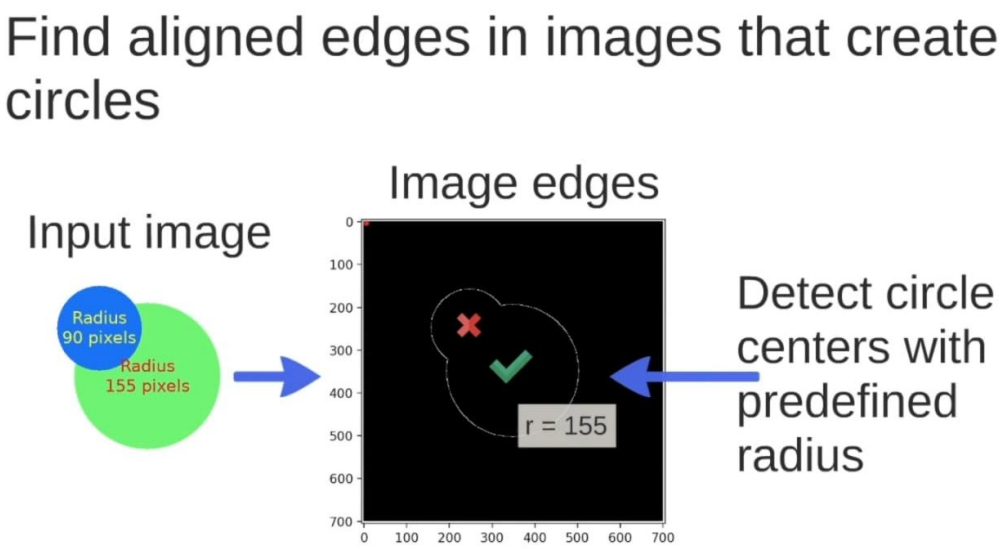
Working Principle:

How the Circles are Detected?

Using Hough Transform the Circles are Detected from the Image of cells:

How Hough's Transform Works?

From an Input Image as shown as below first the edges are Detected and mapped as Shown. *(This Process was Done Previously by Cropping the Image, converting it into Grey Scale and then Blurring it)*



- The input image is first converted into edges so that only the boundaries of circular objects are visible.
- The algorithm assumes a radius and checks all possible center points where a circle could be formed from the detected edges.

- If enough edge points align to form a complete circle, the center and radius are confirmed as a valid detected circle.

Methodology:

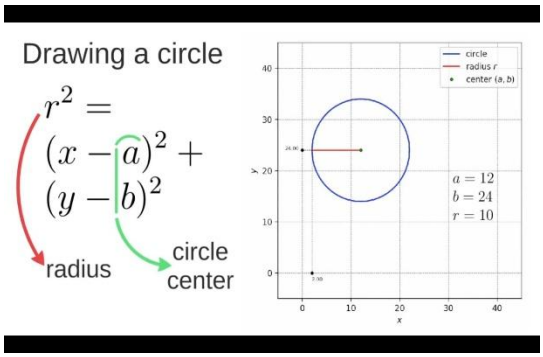


Fig-1

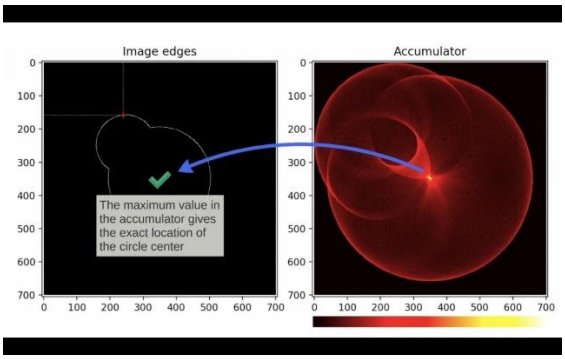


Fig-2

Fig-1 Explains about how a circle can be expressed Mathematically and the same principle is used in the accumulator

Fig -2 Shows how we can map the centre of the Circle using an accumulator based on the pre-defined radius of the Circle. Since the Predefined Radius of the Circle is much greater than the Sub part that is visible above, the smaller circle is not Counted.

In our Current Project we have used the same principle to detect the cells the parameters we used are:

- dp=1.2
  Controls the resolution of circle detection. Lower values give more accurate results but are slower, while higher values are faster but less precise. This value provides a balance between speed and accuracy.

- **minDist** = 16
  Specifies the minimum distance between the centers of

detected circles. Prevents multiple detections of the same cell. If too small, duplicate detections occur; if too large, nearby cells may be missed.

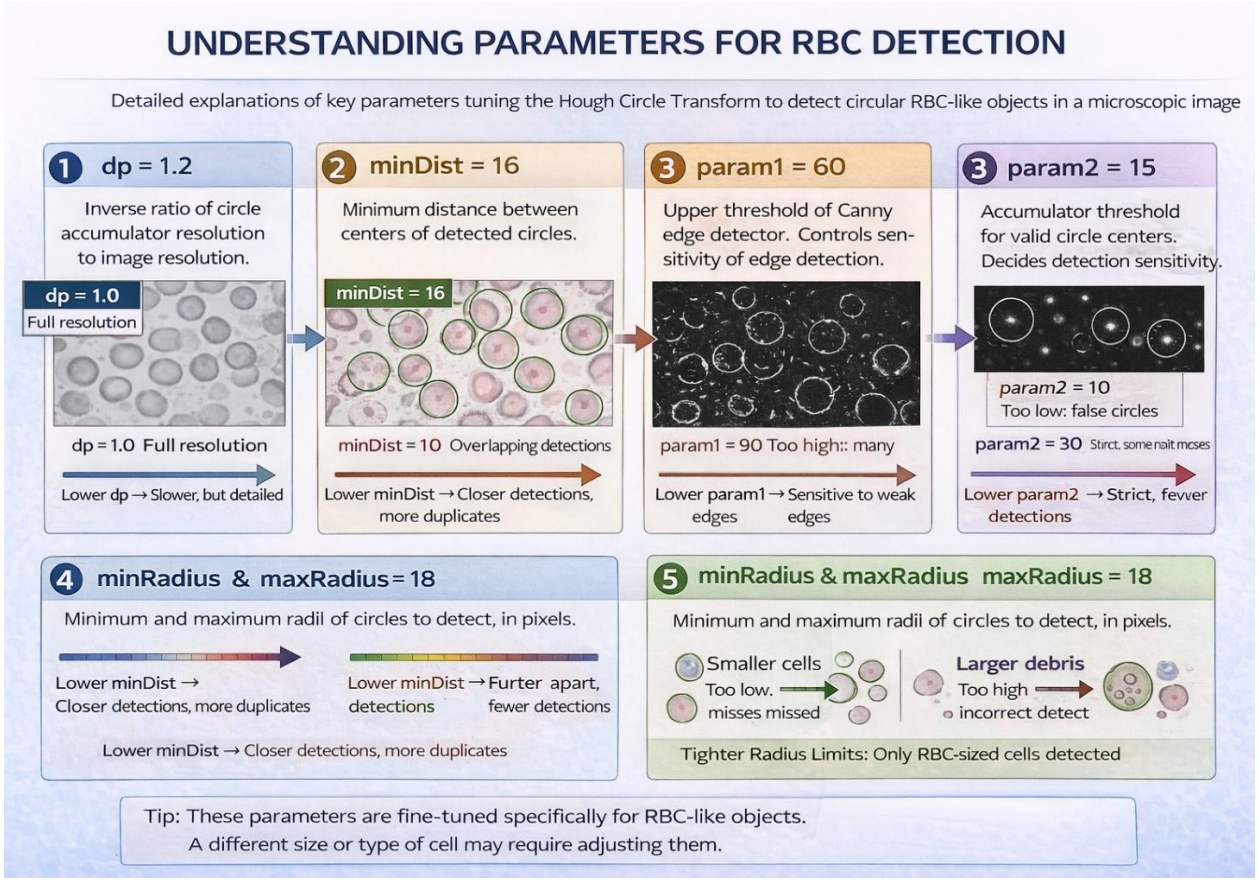
- **param1** = 60
  Represents the higher threshold for edge detection using the Canny method. A lower value detects more edges including noise, while a higher value detects only strong edges. This value helps in identifying clear cell boundaries.

- **param2**=15
  Determines the threshold for circle detection. Lower values make detection more sensitive but increase false positives, while higher values reduce false detections but may miss actual cells. This value ensures a balanced detection.

- **minRadius**=8
  Defines the minimum size of circles to be detected. Helps eliminate small noise or irrelevant objects that are not cells.

- **maxRadius**=18
  Defines the maximum size of circles to be detected. Ensures that only objects within the expected size range of red blood cells are considered.

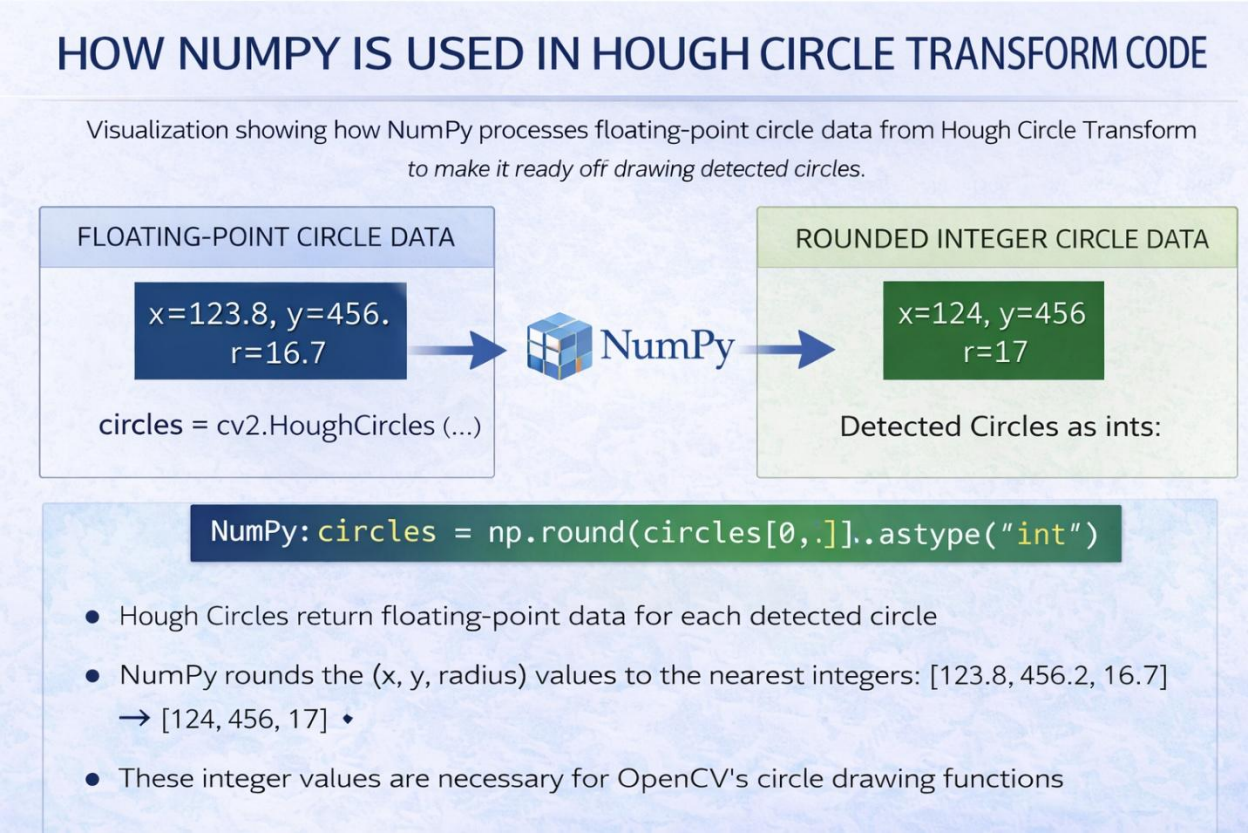
```python
import cv2
# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply median blur to reduce noise
blur = cv2.medianBlur(gray, ksize: 5)

# Hough Circle Transform
circles = cv2.HoughCircles(
    blur,
    cv2.HOUGH_GRADIENT,
    dp=1.2,
    minDist=16,
    param1=60,
    param2=15,
    minRadius=8,
    maxRadius=18
)
```



After the Circles are Detected the Cell Count Is Increased.

How Numpy is Used?

- NumPy is used to process the output of the Hough Circle Transform, which is returned as floating-point values.
- It helps in rounding the detected circle parameters (x, y, radius) to the nearest integer using np.round().
- NumPy converts the data type into integers using .astype("int"), which is required for drawing functions in OpenCV.
- It allows easy indexing and extraction of circle data using array operations like circles[0, :].
- NumPy ensures efficient handling and manipulation of multiple detected circles in a structured array format.

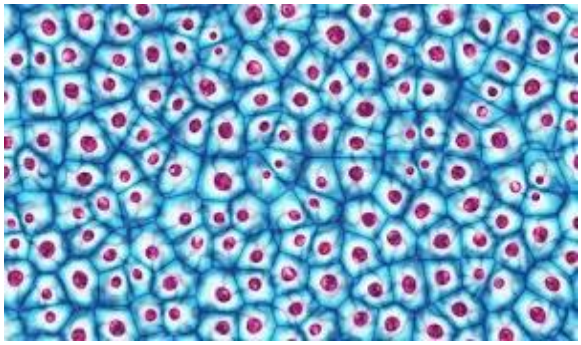


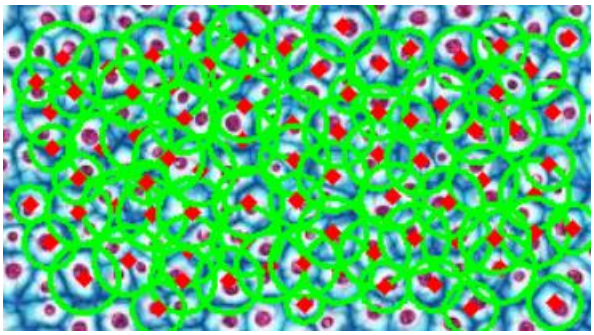
OBSERVATIONS / RESULTS

The system was tested on multiple microscopic images to evaluate its performance. It was observed that the algorithm successfully detected and counted most of the cells present in the images. The detection accuracy was high for clear and well-defined circular cells, while slight deviations were noticed in cases of overlapping or blurred cells. The processing time was minimal, making the system efficient for practical use.

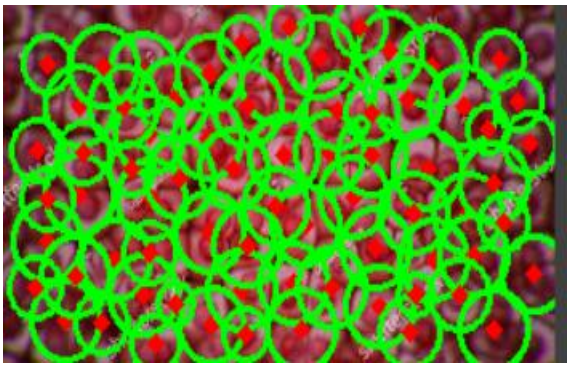
GRAPHS / OUTPUT

The output consists of processed images where detected cells are highlighted using circular boundaries. A comparison between the original and processed images clearly shows the effectiveness of the detection algorithm.

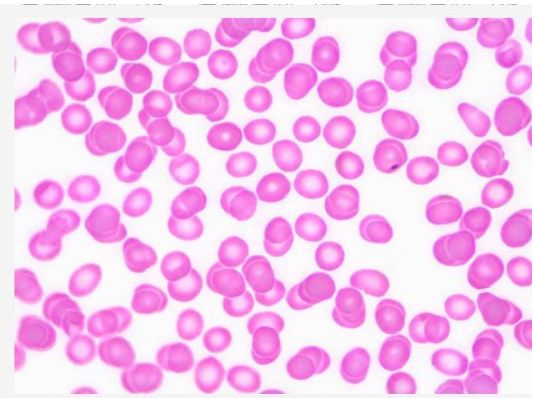


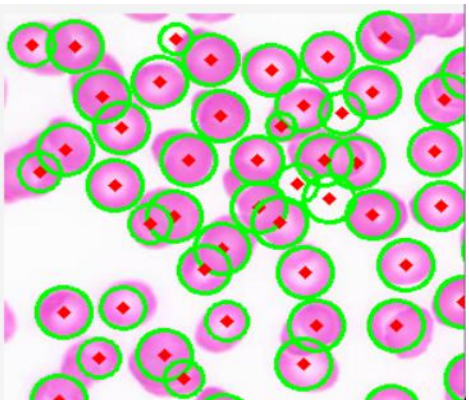


```
C:\Users\siril\PycharmProjects
Estimated RBC Count: 81
```



```
C:\Users\siril\PycharmProjects\P
Estimated RBC Count: 71
```





APPLICATIONS

- Used in medical laboratories for blood cell counting and analysis.
- Helpful in biological and microscopic research for cell studies.

ADVANTAGES

- Reduces manual effort and increases speed of analysis
- Provides consistent and reliable results

LIMITATIONS

- Performance depends on image quality and lighting conditions.

- Difficulty in detecting overlapping or irregularly shaped cells.

FUTURE SCOPE

The system can be improved by integrating machine learning techniques for better accuracy and enabling real-time processing for live cell analysis.

CONCLUSION

The project successfully implements an automated cell detection and counting system using Python and image processing techniques. It provides a fast and efficient alternative to manual counting with satisfactory accuracy.

REFERENCES

1. OpenCV Official Documentation
2. NumPy Library Documentation
3. Standard research articles on image processing and cell detection